Dark Drifts



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

INSTITUTO METRÓPOLE DIGITAL

Manual do Programador

Anderson Araujo

Hiarley Farias

NATAL/RN

JUNHO/2014

1. Introdução

O Dark Drifts e um jogo baseado no jogo já existente, Drifts, a proposta de implementação do jogo foi feita pelas as disciplinas de Estruturas de Dados Básicas 1 e Laboratório de Programação 1 como projeto final do semestre.

Todo o jogo foi desenvolvido usando os conceitos de c e c++ com o auxilio da biblioteca de multimídia SDL 2.0.0.

Esse manual concentra-se no ponto de vista do programador, aqui iremos explicar algumas de nossas técnicas, que foram usadas para o desenvolvimento do código.

2 Classes:
Personagens:

```
 7    private:
 8
 9        char Flag;
10        char Flag_Direcao;
11        int  Raio;
12        int  Position_x;
13        int  Position_y;
14        int  Position_Initial;
15        double  Deslocamento_x;
16        double  Deslocamento_y;
17        int Caminho;
18
19        SDL_Texture *Imagem;
20        int Width;
21        int Height;
22
23
24    public:
25
26        Bola_Vermelha(SDL_Renderer *Janelarenderizada);
27        bool Load_Imagem(SDL_Renderer *Janelarenderizada);
28        void Render_Imagem(SDL_Renderer* Janelarenderizada);
29        char Get_Flag();
30        char Escolhe_Direcao();
31        void Set_Flag(char Novo_Estado);
32        void Set_Position(int x ,int y);
33        void Get_Position(int *x,int *y);
34        void Movimento(double *velocidade_x , double *velocidade_y);
35        void Set_Caminho();
36        void Free();
37        double Escolhe_Velocidade();
```

A mesma implementação de classe pode ser usada praticamente do mesmo jeito para os três personagens, a classe e capaz de carregar e inserir uma imagem na tela, e de movimentar usando uma noção 2d de movimentação.

Na hora da inicialização da classe são definidos algumas variáveis bastante importantes para a classe, como a velocidade em x e y, uma direção para que ele siga o caminho, e uma posição inicial em x.

Movimentação de objeto:

Existe duas possibilidades de caminho para que o objeto siga, pela a esquerda ou pela a direita, isso e escolhido na hora de inicialização da classe.

Conforme a flag de direção foi escolhida ela ira se movimentar sempre seguindo um padrão, se a direção escolhida for a direita ele irá se mover somando a posição inicial em x mais a multiplicação de uma variável de caminho e a variável de velocidade do seu eixo, caso a flag seja a esquerda ela irá diminuir da posição inicial uma multiplicação da velocidade e a velocidade de caminho, a movimentação no eixo y é o mesmo em ambos o caso, uma multiplicação da variável de caminho e uma variável de velocidade.

Existe também uma forma especial de movimentação, quando uma bola verde está grudada na bola jogadora ela se movimenta diferentes das outras, é armazenado a posição x e y do mouse no momento da colisão, na hora de atualizar o objeto que esta grudado novamento e armazenado a posição x e y atual do mouse e então se faz a diferença entre eles para que possa ser incrementado ou decrementado na posição x e y do objeto.

A classe também cumpre a função de testar se a coordenada x e y do objeto estar fora do campo delimitado pela a janela, caso isso aconteça a flag muda para ‘F’, que significa “fora”, que sinaliza para o programa principal que esse objeto não e mais útil.

Circulo:

```
 6    private :
 7        int X , Y , Raio;
 8
 9    public:
10        Circulo( int x, int y, int raio );
11        Circulo(){};
12        int Get_Raio();
13        void Set_Raio(int raio);
14        void Set_X_Y (int x , int y);
15        bool Checar_Colisao(Circulo &A, Circulo &B);
16
17    };
```

A principal função da classe circulo é checar se houve ou não uma colisão entre dois círculos, por meio do calculo da distância, calcula se dois círculos se tocam ou não, retornando um sinal booleano para a jogo.

O Jogo

O jogo se concentra em duas classes bem distintas e que são bastante importantes, uma é a inicia e outra a classe jogo.

Classe Inicia:

Ela e responsável por inicializar alguns dos sub - sistemas do SDL, como o de video e de imagem, ela também cria uma janela do tipo padrão, onde todas as imagens são expostas e trabalhadas, está no seu domínio também criar o menu por onde iremos navegar.

O menu possui quatro opções para ser escolhidas Iniciar Jogo, Como Jogar, Recordes, Créditos,como pode ser pelo as opções, o menu e muito intuitivo.

Classe Jogo:

A classe jogo e responsável por toda a parte do jogo em si, movimentação, projeção de objetos na tela, inicialização de áudio entre outras funções.

A classe se divide em algumas funções principais, escolher objeto que será projetado na tela, criar um objeto, checar se houve colisão com outro objeto, liberar objeto desnecessários.

Escolher um objeto:

Por meio de uma variável global que contabilíza o número de interação do programa é escolhido um objeto, que por meio da função rand() mod 3, gera um número aleatório que quando junta com aritmética modular reduz o numero a um conjunto de resíduos que varia de 0 á 2, que e a quantidade exata de possibilidade de objetos a serem escolhidos.

Depois de ser escolhido a bola a ser inserida, o programa cria um objeto alocado dinamicamente para o tipo definido incrementa uma variável global de controle, e depois o guarda em um vector encadeado.

Atualização da Imagens

Na hora de atualizar as imagens, cada elemento de cada vector será atualizado, irá se chegar se existe elementos inúteis e checar se houve algum tipo de colisão. As classes objetos tem funções especificas para a movimentação, então o programa só é responsável por chamar essas funções.

```
Bolaverde_it->Set_Caminho();
Bolaverde_it->Render_Imagem(Janelarenderizada);
```

. Depois de atualizar todos os elementos o jogo agora testa se algum dos círculos se tocam, as únicas colisões testadas pelo o programa são quando a bola do jogar toca uma outra bola ou quando uma bola grudada na bola jogadora toca em outra bola, existem três tipos de possíveis colisões, com bolas verdes , vermelhas e púrpuras.

Colisões com bolas verdes

Se houver uma colisão com uma bola verde, essa bola verde será adicionada a um novo vector e marcada com fora no vector anterior, depois sera depois será incrementado uma variável de controle.

Colisões com bolas vermelhas

Se houver uma colisão com uma bola vermelha, uma vida será decrementada do seu total, caso haja bolas grudadas na bola jogadora elas serão liberadas, e o vector que as bolas grudadas estão grudadas será esvaziado .

Colisões com bolas púrpuras

Se houver uma colisão com uma bola púrpura, será inspecionado quantas bolas estão grudadas na bola jogadora, caso houver de três ou mais bolas uma pontuação será creditado e armazenado numa variável de controle, se o critério não for respeitado então todas as bolas serão liberadas.

Remoção de Bolas

Depois de checar todas de atualizar todos os objetos e checar todas as colisões, o jogo agora irá remover dos vector todos as bolas que tiverem suas flag como ‘F’, a cada bola que for removida a variável que é responsável por controlar o número de elementos do conjunto é decrementado.